

Duo Chat Runbook

Overview

This page explains how to investigate Duo Chat issues on production.

Quick Links

- Dashboard:
 - Kibana based dashboard
 - Prometheus based dashboard ... TBD
 - Evaluation by Prompt Library
 - Error budget dashboard
- GitLab-Rails:
 - GitLab Rails GraphQL log (Chat)
- GitLab-Sidekiq:
 - GitLab Sidekiq worker log
 - GitLab Sidekiq LLM log
 - LLM Completion worker
 - Duo Chat specific error codes
- Redis:
 - Redis Chat Storage (To be decommissioned)
- AI Gateway:
 - AI Gateway log (Duo Chat)
 - AI Gateway log (Request errors)
 - AI Gateway metrics (Chat SLI)
 - Runway AI Gateway metrics
- Anthropic APIs:
 - SLI/SLO dashboard ... TBD
 - Anthropic API Status Page (NOTE: This displays entire Anthropic system operational status, which might be unrelated to our workload)
- Others:
 - Kibana
 - Prometheus
 - Tableau (Duo Chat usage)
 - Tableau (Unit primitives)

Agentic Duo Chat

If you are troubleshooting issues with **Agentic Duo Chat**, please refer to the Agentic Duo Chat Runbook instead.

Before starting investigation

NOTE: Do **NOT** expose customer's RED data in public issues. Redact them or make a confidential issue if you're unsure.

Before starting the investigation, please collect the following information:

- User name who encountered the bug (e.g. @johndoe)
- Determine if the user is using Agentic Chat or Duo Classic Chat
- What happened (e.g. User asked a question in Duo chat and saw an error code A1001)
- When it happened (e.g. Around 2024/09/16 01:00 UTC)
- Where it happened (e.g. VS Code, Web UI)
- How often it happens (e.g. It happens everytime)
- Steps to reproduce (e.g. 1. Ask a question “xxx” 2. Click ...)
- Whether we can enable Expanded AI logging and retry the bug so that we can collect process-level logging.
- (V2 Chat Agent only) Whether the bug can be reproduced with Chat Agent V1 as well. See how to disable the feature flag for a specific user.
- (Agentic Chat only) Session ID. The session ID can be found in the three ellipses located at the top of the Duo Chat window
- (Self-managed only) GitLab version (e.g. v17.4)
- (Self-managed only) GitLab host name (e.g. my-org.gitlab.io)
- (Self-managed only) whether they use GitLab-managed AI Gateway or self-managed AI Gateway (If they use custom models, it’s likely latter.)
- A link to a Slack discussion, if any.

and create an issue in the GitLab Issue tracker and ping @gitlab-org/ai-powered/duo-chat.

Logs

Log links for various environments can be found here.

Different deployments use different indexes. The following indexes are most helpful when debugging Duo Chat:

- AI Gateway logs are in the `pubsub-mlops-inf-gprd-*` index
- GitLab Rails Sidekiq logs are in the `pubsub-sidekiq-inf-gprd*` index
 - All LLM Sidekiq traffic is sent to a single Sidekiq shard, filtering on `json.shard.keyword: "ai-abstraction-layer"` will only return `ai-abstraction-layer` traffic.
 - When searching this index, filtering on `json.subcomponent : "llm"` ensures only LLM logs are returned
- GitLab Rails logs are in the `pubsub-rails-inf-gprd-*` index

Chat GraphQL request logs for a user can be found with the following Kibana query in the Rails (`pubsub-rails-inf-gprd-*`) index:

```
json.meta.user : "your-gitlab-username" and json.meta.caller_id  
: "graphql:chat"
```

If you find requests for a user there but do not find any results for them using a Kibana query in the Sidekiq (`pubsub-sidekiq-inf-gprd*`) index:

```
'json.meta.user' : "username-that-received-error" and  
json.subcomponent : "llm"
```

That probably indicates a problem with Sidekiq where the job is not being kicked off. Check the [#incidents-dotcom](#) to see if there are any ongoing Sidekiq issues. Chat relies on Sidekiq and should be considered “down” if Sidekiq is backed up. See Duo Chat does not respond or responds very slowly below.

AI Abstraction Layer Sidekiq Traffic

Duo Chat requests and some Duo experimental features go through an isolated `urgent-ai-abstraction-layer` Sidekiq shard which provides a centralized platform to handle asynchronous jobs for our external LLM inferences. As part of the AI Framework’s resilience objective, we’ve migrated our Sidekiq traffic onto one single shard to separate LLM requests from the entire Gitlab’s sidekiq jobs.

To find only Duo traffic, you can click on the `pubsub-sidekiq-inf-*` Elastic Search.

- Filter the logs by selecting `json.shard.keyword: "urgent-ai-abstraction-layer"` to limit logs coming from our respective Sidekiq containers.

Important Feature Category information: Sidekiq feature category: `urgent-ai-abstraction-layer` GKE Deployment: `gkeDeployment: 'gitlab-sidekiq-urgent-ai-abstraction-layer-v2'` Queue Urgency: `throttled`

See this [issue](#) for more information.

Tracing requests across different services

We utilize a `correlation_id` attribute to track and correlate log entries across different services. This unique identifier serves as a key to tie together logs for different systems and components.

In Duo Chat case, mainly these components are involved:

- GitLab-Rails ... It provides the GraphQL API interface for frontend clients (VS Code extension, WebUI). Since it invokes a sidekiq job to perform the actual process as *a separate process*, this correlation ID can’t be used for tracing the actual processes.
- GitLab-Sidekiq ... It performs the main process of Duo Chat request. If you’re debugging Duo Chat functionalities, you need to grab a correlation ID in the Sidekiq logs.
- AI Gateway ... It performs the AI related process of Duo Chat request. You can find a correlated logs from the Sidekiq’s correlation ID.

Here is an example of how to find correlated logs in the AI Gateway:

- Access the `pubsub-mlops-inf-gprd-*` index:

- Filter for the logs with `json.jsonPayload.correlation_id : <correlation_id`
- *optional*: Click on the expanded logs icon and select “Surrounding documents” to view logs within relative same time stamp.

How to determine global user ID for a user

When troubleshooting requests from self-managed users on the AI Gateway, it may be helpful to find their global user ID to narrow down requests. They should run this command on their instance rails console:

```
u = User.find_by_username(<USERNAME>)
Gitlab::GlobalAnonymousId.user_id(u)
```

Then you can filter by `json.jsonPayload.gitlab_global_user_id` to see requests from that specific user.

You can also attempt to figure it out if you know the `gitlab_host_name` and approximate timestamp.

1. Go to AI Gateway log (Duo Chat)
2. Filter by the `json.jsonPayload.gitlab_host_name`
3. Narrow down the request by the timestamp given by the customer
4. Look at the requests in the given time period and try to determine a `gitlab_global_user_id` that fits.

This process involves guesswork, so it is best to ask the customer directly.

Extra Kibana links

You can find other helpful log searches by looking at saved Kibana objects with the `group::ai_framework` tag.

- AI Gateway error rates and response statuses
- AI Gateway Error Rate

Duo Chat specific error codes

All of GitLab Duo Chat error codes are documented here. The error code prefix letter can help you choose which Kibana logs to search.

Error Code Layer Identifier

Code	Layer
M	Monolith - A network communication error in the monolith layer.
G	AI Gateway - A data formatting/processing error in the AI gateway layer.
A	Third-party API - An authentication or data access permissions error in a third-party API.

Debugging Error Codes A1000-6000

When you receive an error code starting with ‘A’, there’s an error coming from the AI Gateway.

This can mean that the AI Gateway service itself is erroring or that a third-party LLM provider is returning an error to the AI Gateway.

1. Check for any ongoing outages with our third-party LLM providers:
 - Anthropic API Status
 - Google Cloud Platform Status
2. Use the Grafana Dashboard to determine the overall impact. The **Aggregated Service Level Indicators** () metric on that page indicates what percentage of users/requests are encountering errors.
3. Track down the specific error
 - Search for any Chat requests with errors for the user in the Sidekiq logs (pubsub-sidekiq-inf-gprd-*): `json.meta.user : "username-that-received-error"` and `json.subcomponent : "llm"` and `json.error : *`. The log line with the `json.error` value that matches what the user is seeing is what you want to use. Copy the `json.correlation_id` value.
 - Search for the request in the AI Gateway logs (pubsub-mlops-inf-gprd-*): `json.jsonPayload.correlation_id : "correlation_id-from-last-result"`
 - The `json.payload.Message` value in the AI Gateway log results should indicate what error message we are receiving from Anthropic, if any.

Debugging Error Codes M3002 - M3004

The issue most likely exists within the Monolith. Look for this error in the Sidekiq logs.

1. Filter out json logs to the subcomponent `llm.log` with `json_subcomponent.keyword : "llm"`
2. Filter out error codes with the specific M error code with `json.error_code : "<error_code>"`
3. Check to see issue occurs with a specific user with `json.meta.user : "<user_name>"`
4. Make sure the “Calendar Icon” has the query active for the relevant issue. The default time stamp is 15 minutes realtive from the current date.

The following should provide enough information to boil down error logs for a specific user, error code, and all relevant llm logs that follow underneath our AI logs. Some common issues that cause the following error are:

1. Rails application having issue with an access check for the current resource.
2. Duo features aren’t enabled with the group or project.

Debugging Error Codes M3005

The following error is pretty straightforward for reasoning. The M3005 error code indicates that the user is requesting a chat capability that belongs to a higher add-on tier, which the user does not currently have access to. This error occurs when attempting to use features or functionalities that are not included in the user’s current subscription level or plan.

Please report this issue to the development team [#g_ai_framework#](#). It most likely indicates an issue with the access control for guarding unit primitives.

Debugging Error Codes M4000

The following all relate towards a slash command issues.

Slash Command	Tool	SME Slack Channel
/troubleshoot		#f_ci_rca
/explain		#g_code_creation
/tests		#g_code_creation
/summarize_comments		#f_plan_ai
/refactor		#g_code_creation
/vulnerability_explain		#f_ci_rca

Debugging Error Code G3001

This error code occurs when GitLab Duo Chat is not available for a certain subscription.

Duo Chat does not respond or responds very slowly

This could be caused by an issue with Sidekiq queues getting backed up. First, check the GitLab status page to see if there are any reported problems with Sidekiq or “background job processing”. Then, check this dashboard. If you see that ‘scheduling time for the completion worker’ values are much higher than normal, it indicates the Sidekiq backup may be the problem.

Expanded AI logging

WARNING: DO NOT ENABLE FOR CUSTOMERS. GitLab does not retain input and output data unless customers provide consent through a GitLab Support Ticket.

We do allow the option to enable enhanced ai logging by enabling the `expanded_ai_logging` feature flag. The flag will allow you to see input and output of any of the following AI tools.

To enable expanded AI logging, access the **#production** Slack channel and run the following command.

```
/chatops gitlab run feature set --user=$USERNAME expanded_ai_logging true
```

After the the **expanded_ai_logging** feature flag is enabled for a user, you view the user input and LLM output for any the GitLab Duo Chat requests made by the user. We've extended the support to AI Gateway as well, so you can get a process-level logging, including actual request parameters and LLM response in the AI Gateway logs.

Tips:

- To trace the request across different services, use correlation-id.
- We only need to enable the flag while we reproduce the bug on production. After we sampled a couple of problematic requests, we can disable the flag again and continue examining the logs.

Agentic Chat troubleshooting

For help troubleshooting Agentic Chat, please refer to these guides in the handbook for Duo Chat and Agent Foundations

When problem is only identified on staging

Here are the log links for staging:

- Rails
- Sidekiq

Make sure you have access to Duo Chat on staging. If not, request access to Duo Enterprise on the **#g_provision** Slack channel (for non-production environments only).

If there is a problem only on staging but not production, the **env** variables may be at fault. Compare the default **env** variables from staging and production to see if you can spot a relevant difference.

How to identify IDE-specific problems

When a customer reports a problem with Duo Chat in the IDE, it can be difficult to tell if the problem is IDE-specific or not.

The first step is to have the customer test their query on the web version of Duo Chat. If they have the same problem on web, it is not IDE-specific.

Then, they can perform these steps to determine if it is a backend (AI Gateway) problem, or client-side on the editor:

1. Ask the question in web
2. Observe it works

3. Run the `/reset` command on the web version
4. Ask the same question in the IDE plugin
5. Go back to the web and refresh the page.
6. Check if there is a response there

If the response does not show up on web, it is likely an AI Gateway problem. If the response does show up on web, it is likely a client-side IDE problem.

When a Duo Chat specific error code happened on self-managed GitLab

When a Duo Chat specific error code happened on self-managed GitLab, the following logs are helpful for further investigation:

- LLM log
 - To get the full details, `expanded_ai_logging` feature flag needs to be enabled. Please see the admin doc for more information.
- Sidekiq log

Collect the log from the timestamp that the user reproduced the error code. 5-10 minutes of timerange should be enough.

After we've collected the log, we do:

1. Filter the `llm.log` by the error code (LLM log outputs the error code as-is). Extract the correlation-id in the same log line.
2. Filter the `llm.log` and `sidekiq.log` by the extracted correlation-id. This gives us the details of the process flow, which is crucial to identify where the thing went wrong.

Rate Limits

Duo Chat has the following rate limits:

- AI Action Rate Limit: 160 calls per 8 hours per authenticated user
 - This limit applies to GraphQL `aiAction` mutations
 - When exceeded, returns error code A1001 with message “This endpoint has been requested too many times. Try again later”
 - Configured via `application_settings.ai_action_api_rate_limit`
 - Can be monitored in GitLab Rails error rates dashboard

When users hit rate limits:

1. Check current rate limit usage:

```
# In Rails console
user = User.find_by_username('username')
Gitlab::ApplicationRateLimiter.throttled?(:ai_action, scope: [user], peek: true)
```

2. For temporary relief, rate limits can be reset:


```
# In Rails console  
Gitlab::RateLimitHelpers.new.reset_rate_limits(:ai_action, user)
```

3. For persistent issues, consider:

- Reviewing usage patterns to identify potential abuse
- Adjusting the global rate limit via application settings if needed
- Adding user to allowlist if legitimate high usage case